

PROJECT MANUAL · V2.1 · 2026-07-20

Ai102

A local-first AI presentation generator that drafts, designs, **reviews, and auto-fixes** slide decks — free to run on open models, with real OAuth sign-in.

Repository	github.com/Suppakris/Ai102
Live deployment	ai102-psi.vercel.app
Based on	ALLWEONE's presentation-ai (fork, customized for coursework)
License	MIT
Framework	Next.js 16 · React 19 · TypeScript
Replaces	Manual V1.0 (2026-07-16) — see Section 2 for everything that changed

Table of Contents

1. Project Overview
2. What Changed Since Manual V1.0
3. What's Different From Upstream
4. Features
5. Tech Stack
6. Getting Started
7. Environment Variables
8. Database Setup (SQL Migrations)
9. Running on Docker
10. Team Workflow
11. Usage Guide
12. Project Structure
13. Known Issues
14. Contributing
15. License

1. Project Overview

Ai102 is a local-first AI presentation generator, a customized build derived from ALLWEONE's open-source presentation-ai. Text generation runs on a local (or tunneled) **Ollama** model by default — no cloud LLM subscription required — with OpenRouter available as an optional, admin-funded upgrade.

The app takes a topic (or a source document: PDF, Word, Excel, CSV, or plain text), generates an editable outline, then produces a full slide deck with free AI-generated images, editable themes, rich text, charts, and PowerPoint export.

The headline feature on top of generation is the **AI Deck Review**: an auditor model scores every deck on clarity, design, and content accuracy, fact-checks its claims, and — with one click — **auto-fixes** failing decks by rewriting the flagged slides (with undo).

Sign-in is real OAuth (GitHub required; Google and Discord optional). All core features are free for every signed-in user.

2. What Changed Since Manual V1.0

V1.0 (written 2026-07-16) documented an earlier state of the code and flagged three "documentation vs. code" discrepancies. All three are now resolved, and several major features landed after it was written. If you read V1.0, discard these claims:

Manual V1.0 said	Now (V2.0)
Auth is disabled; a demo-user stub makes everyone admin	Real OAuth via Auth.js/NextAuth v5: GitHub required, Google/Discord optional. New sign-ins default to <code>role: USER</code> ; admin is granted via the DB (Section 7).
OpenRouter path in Docker is dead code	Wired and functional. Set <code>OPENROUTER_API_KEY</code> and a paid model group (6 preset cloud models) appears in the model picker for all users.
Image generation defaults to FAL (paid)	Free by default via Pollinations.ai (no API key). FAL remains an optional admin-gated upgrade.
The slide audit agent is unbuilt (no API route)	Shipped as AI Deck Review: <code>POST /api/presentation/review-deck</code> , a Review button in the editor, one-pass Auto-fix with undo, bilingual (EN/TH) feedback, plus test harnesses <code>pnpm review:test</code> and <code>pnpm review:bash</code> .
Only PDF can be attached as source material	PDF, Word (.docx), Excel (.xlsx/.xls), CSV, and text/Markdown — all parsed in the browser, never uploaded.
No marketing page; app drops straight into /presentation	A landing page greets signed-out visitors; signed-in users go straight to the dashboard. The app also gained a light/dark toggle, a navigation menu with sign-out, and an electric-blue/cyan visual identity.
Migrations: only <code>0_init</code> existed	Three migrations. <code>20260718150000_add_tenancy_and_agent_threads</code> closes a gap where multi-tenancy tables existed in the schema but not in migrations (fresh Docker DBs were missing tables).

3. What's Different From Upstream

- **Auth is real OAuth** (`src/backend/auth.ts`, Auth.js/NextAuth v5 + Prisma). GitHub is required; Google and Discord are optional — each provider's sign-in button renders only when its credentials are configured. All three are free. New sign-ins default to `role: USER`.

Test social sign-in only on the real deployed domain, never a Vercel PR preview URL. OAuth apps support exactly one callback URL (production), so preview-URL sign-ins always fail with `error=Configuration`. Expected, not a bug.

- **Text generation defaults to Ollama, with OpenRouter as an optional paid upgrade** (`src/lib/model-picker.ts`). Requests resolve to a model served from `OLLAMA_BASE_URL`'s native API unless `OPENROUTER_API_KEY` is set, which unlocks 6 preset cloud models for all users (the key's owner pays per token).
- **Image generation is free by default** — Pollinations.ai (no API key) everywhere images are generated; FAL (Flux) is an optional admin-gated paid upgrade.
- **Backend logic lives under** `src/backend/` — db, auth, tenants, rate limiting, the image queue, and the LangGraph agent. See ARCHITECTURE.md.

4. Features

Core Functionality

- AI outline generation, then full slide generation, on a local Ollama model
- Use your own documents as source material: drop a **PDF, Word (.docx), Excel (.xlsx/.xls), CSV, or text/Markdown** file into the prompt box — parsed entirely in the browser (never uploaded), trimmed to the model's context budget
- Editable outlines · real-time generation · auto-save
- In-editor chat agent (LangGraph + Postgres memory): edit layout/background, replace/regenerate images, create/delete slides, switch themes, web search (needs `TAVILY_API_KEY`)

AI Deck Review & Auto-fix HEADLINE FEATURE

- One-click **Review** in the editor header: scores 0–10 on clarity, design, content accuracy, with a pass / needs-revision verdict
- **Claim audit**: every verifiable factual claim is marked Supported / Unsupported / Unverifiable — the presentation's own prompt and outline are passed as source context, so claims that trace back to them are genuinely verified
- **Auto-fix**: a failing deck gets one corrective AI pass — unsupported claims removed or softened (never given invented backing), the rewrite is re-reviewed and applied to the editor, with one-click Undo. All-or-nothing: a rewrite that can't be parsed back into slides leaves the deck untouched
- Feedback in the deck's own language (Thai decks get Thai feedback); sparse decks get clarifying questions instead of blind scores
- Backend: `reviewSlides()` / `reviewAndRevise()` in `src/backend/ai/reviewSlides.ts`, served by `POST /api/presentation/review-deck` (session auth + rate limit), schema-enforced JSON output
- Test harnesses: `pnpm review:test` and `pnpm review:bash` (adversarial decks)

Design & Customization

- Built-in themes, custom theme creation, PPTX theme import; picking a theme no longer recolors the page chrome

Presentation Tools

- Present from the app · public share links · PPTX export · charts/infographics/embeds · Plate rich-text editor · inline comments

Images

- Pollinations.ai (free, default) · FAL Flux (optional paid) · Unsplash/Pixabay/Giphy stock · Google Custom Search

5. Tech Stack

Category	Technologies
Framework	Next.js 16, React 19, TypeScript
Styling	Tailwind CSS v4 (electric-blue/cyan token palette, light + dark)
Database	PostgreSQL (Supabase in prod, Docker Postgres locally) via Prisma ORM + committed SQL migrations
Auth	Auth.js / NextAuth v5 — GitHub (required), Google + Discord (optional)
Text generation	Ollama (default, native API) or OpenRouter (optional paid), via LangChain + LangGraph
Image generation	Pollinations.ai (free default), FAL Flux (optional paid)
Queue	BullMQ + Redis (image generation moves off the web process when <code>REDIS_URL</code> is set)
UI / Editor	Radix UI · Plate Editor (platejs)
Lint/format	Biome (canonical)

6. Getting Started

Prerequisites

- pnpm (pinned to `pnpm@11.1.3`)
- A PostgreSQL database (Supabase, Neon, or the Docker Postgres from Section 9)
- **Ollama** running locally or reachable via `OLLAMA_BASE_URL`, with a model pulled (e.g. `ollama pull qwen2.5:7b` — the review feature's reference model)
- GitHub OAuth app credentials (free) for sign-in

Installation

```
git clone https://github.com/Suppakris/Ai102.git
cd Ai102
pnpm install                # postinstall runs prisma generate
cp .env.example .env        # fill in DATABASE_URL + auth vars (Section 7)
pnpm db:migrate:deploy      # apply SQL migrations (Section 8)
ollama pull qwen2.5:7b
pnpm dev
```

The app runs at `http://localhost:3000`. Signed-out visitors see the landing page; sign in with GitHub (or Google/Discord if configured) to reach the `/presentation` dashboard.

7. Environment Variables

`.env.example` is the source of truth. The notable ones:

Variable	Required?	Purpose
<code>DATABASE_URL</code>	Required	Postgres connection string. For Supabase use the Transaction pooler (port 6543) — serverless functions exhaust Session/Direct pooler connections fast.
<code>NEXTAUTH_SECRET</code> / <code>NEXTAUTH_URL</code>	Required	Auth.js session signing + canonical URL.
<code>GITHUB_CLIENT_ID</code> / <code>GITHUB_CLIENT_SECRET</code>	Required	GitHub OAuth app (free). The app won't boot in production without auth vars.
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	Optional	Adds a Google sign-in button. Remember to add the callback URL under Authorized redirect URIs in Google Cloud Console.
<code>DISCORD_CLIENT_ID</code> / <code>DISCORD_CLIENT_SECRET</code>	Optional	Adds a Discord sign-in button.
<code>OLLAMA_BASE_URL</code>	Optional	Remote/tunneled Ollama (e.g. ngrok) instead of localhost. Note: ngrok URLs rotate on restart — update this var (and redeploy) when the tunnel restarts.
<code>OLLAMA_DEFAULT_MODEL</code>	Optional	Default model. Set <code>qwen2.5:7b</code> in production — the small fallback produces degenerate reviews. Must be set wherever the app runs (e.g. Vercel), not just locally.
<code>OLLAMA_NUM_CTX</code>	Optional	Context window (default 8192). Ollama's own 2048 default truncates this app's prompts.
<code>OPENROUTER_API_KEY</code>	Optional	Unlocks 6 preset cloud models for all users; key owner pays per token.
<code>FAL_API_KEY</code>	Optional	Paid image upgrade (admin-gated). Images work free without it via Pollinations.ai.
<code>REDIS_URL</code>	Optional	Moves image generation onto the BullMQ worker; without it jobs run inline.

TAVILY_API_KEY	Optional	Web search for outline generation and the chat agent.
UPLOADTHING_TOKEN, UNSPLASH_ACCESS_KEY, GOOGLE_CUSTOM_SEARCH_API_KEY + SEARCH_ENGINE_CX	Optional	Image storage / stock photos / Google image search.

Admin access

New sign-ins default to `role: USER`. To grant admin (paid FAL models, editing system themes):

```
pnpm db:studio    # open the User table, set role = ADMIN
-- or SQL:
UPDATE "User" SET role = 'ADMIN' WHERE email = 'their@email.com';
```

8. Database Setup (SQL Migrations)

The schema is managed with committed SQL migrations in `prisma/migrations/`:

```
pnpm db:migrate:deploy  # apply all pending migrations
pnpm db:migrate:dev     # after editing schema.prisma: generate + apply a new migration
pnpm db:studio          # browse the DB
```

Commit each generated folder — the SQL file is the schema change. `pnpm db:push` exists for throwaway prototyping only; never use it on a shared database.

Current migrations: `0_init` → `20260714110000_add_slide_audit` → `20260718150000_add_tenancy_and_agent_threads`.

Existing databases created with `db push` (including the production Supabase DB) already contain all tables. Baseline them once instead of re-applying — otherwise `migrate deploy` will try to create tables that already exist:

```
pnpm exec prisma migrate resolve --applied 0_init
pnpm exec prisma migrate resolve --applied 20260714110000_add_slide_audit
pnpm exec prisma migrate resolve --applied 20260718150000_add_tenancy_and_agent_threads
```

After baselining, `pnpm db:migrate:deploy` reports "No pending migrations" and future migrations apply normally.

No seed step required — signing in creates the user record via the Prisma adapter.

9. Running on Docker

The whole stack — front-end + back-end + Postgres + Redis + the image-generation worker — runs on Docker with nothing on the host except Docker itself:

```
docker compose up --build
```

Boot order is automatic: Postgres becomes healthy → a one-shot `migrate` container runs `prisma migrate deploy` → the app starts at `http://localhost:3000` alongside a separate BullMQ worker container consuming the Redis-backed image queue.

LLM access from inside the container

- **Ollama on the host** (default): compose points `OLLAMA_BASE_URL` at `host.docker.internal:11434`.
- **OpenRouter**: put `OPENROUTER_API_KEY=sk-or-...` in a `.env` next to `docker-compose.yml`. (This path is now fully wired into the app — V1.0's "dead code" warning no longer applies.)

Auth inside Docker

Set real `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET` (and optionally the Google/Discord pairs) in the same `.env`. With the placeholder defaults the stack boots and serves pages, but sign-in won't work.

Postgres data persists in the `db-data` volume; `docker compose down -v` wipes it.

10. Team Workflow

	Track 1 — Prompt Testing	Track 2 — DevOps
Owns	Prompt templates, agent behavior, model/provider selection, output quality	Docker, database migrations, environment config, deployment
Files	<code>src/backend/agent/**</code> , <code>src/backend/ai/**</code> , <code>src/lib/presentation/*prompt*</code> , <code>src/lib/model-picker.ts</code> , <code>src/app/api/presentation/**</code>	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>prisma/migrations/**</code> , <code>prisma/schema.prisma</code> , <code>.env.example</code> , Vercel settings
Test loop	Judge review/generation quality with the shipped harnesses: <code>pnpm review:test</code> and <code>pnpm review:bash</code> (local Ollama free, or <code>--openrouter</code> for reproducible cloud runs)	<code>docker compose up --build</code> must boot a working app from a clean checkout; schema changes ship as SQL migrations via <code>pnpm db:migrate:dev</code>
PR scope	Prompt/agent changes only	Infra changes only

1. A schema change is a DevOps-track change even if a prompt PR needs it — split it out.
2. Prompt changes are judged by harness results, not one lucky generation.
3. Both tracks branch from `main` and merge via PR.

11. Usage Guide

1. **Sign in** at the deployed site (or `pnpm dev` locally) with GitHub, Google, or Discord.
2. **Create a deck** from the dashboard: describe a topic in the prompt box — or drop in a PDF/Word/Excel/CSV/text file as source material — pick slide count, language, and theme, then generate. Review and edit the outline before rendering slides.
3. **Edit** in the editor: change themes (the `√` next to the title opens the document menu; `≡` opens app navigation), edit text directly, or ask the chat agent for changes.
4. **Review**: click **Review** in the header. You get scores, written feedback, and a claim audit. If the deck fails, click **Auto-fix deck** — the AI rewrites flagged slides and re-reviews; use **Undo** if you don't like the result. A deck built on fabricated claims may honestly stay "needs revision" — the AI removes fake claims but cannot invent real data.
5. **Ship it**: export to `.pptx`, present from the browser, or share a public link.

If the header shows "**Backend issue detected**", the Ollama server is unreachable — usually the ngrok tunnel restarted and `OLLAMA_BASE_URL` needs the new URL (then redeploy).

12. Project Structure

See ARCHITECTURE.md for the full split. Quick map of `src/`:

```
src/
├─ app/                Next.js App Router routes
│  ├─ page.tsx         landing page (signed-out) / redirect to dashboard
│  ├─ presentation/    dashboard, create flow, generation view, editor ([id]/)
│  ├─ share/           public read-only share view
│  ├─ api/             route handlers (incl. presentation/review-deck)
│  └─ _actions/        Server Actions → delegate to src/backend/**
├─ components/notebook/ slide-outline UI, theming, editor plugins
├─ components/presentation/ app chrome: header (menu/avatar/toggle), review button,
│                             edit panel, present mode
├─ backend/            db.ts · auth.ts (NextAuth v5) · tenant.ts · rate-limit.ts
│  ├─ ai/reviewSlides.ts review + auto-fix engine
│  ├─ queue/           BullMQ image queue (Pollinations/FAL)
│  └─ agent/           LangGraph editing agent + tools
└─ lib/               model-picker.ts · presentation/ (themes, prompts,
                      pdf-extract.ts, office-extract.ts) · observability/
```

13. Known Issues

- **You can't upload your own images into slides** — slide images come from the AI image generator (free Pollinations by default) or stock/search sources. There is currently no working way to add a picture from your own device; an upload option is on the wishlist but not built.
- **Slide generation can occasionally glitch** — deck content is written live by an AI model, so a run can sometimes produce a broken layout, a half-filled slide, or fail partway through. This is not data loss: regenerating the slide (or the whole deck) usually fixes it. Reliability also depends on the shared Ollama server being reachable at that moment.
- **Present mode still has UI rough edges** — when presenting a deck full-screen, some slides may render with visual glitches (elements misplaced, scaled oddly, or styled differently than in the editor). The content itself is unaffected — it's a display-layer issue on the presentation view that's still being polished.
- **Document/RAG search is scaffolded, not implemented** — no vector search is wired into the agent; "chat with your document" doesn't work yet. (Attaching documents as generation source material *does* work — that's a different, shipped path.)
- **New sign-ins default to `role: USER`** — FAL paid models and system-theme editing stay hidden until admin is granted (Section 7). All core features work at USER.
- **Review history is not persisted** — reviews display in the dialog only; a stored history is the agreed next feature.
- **Shared-Ollama single-file line** — one Ollama server serializes concurrent generation requests; heavy simultaneous use queues up.
- **Legacy .doc (Word 97–2003) is not supported** as source material (modern .docx only); scanned PDFs need selectable text.
- Both Biome and Prettier configs exist; Biome is canonical.

14. Contributing

Workflow

1. Fork, clone, `pnpm install`, branch (`feature/your-feature-name`)
2. Make changes; run `pnpm type` and `pnpm check` before pushing
3. Open a PR with a clear title and description; address review feedback

Commit prefixes

`feat:` new features · `fix:` bug fixes · `docs:` documentation · `style:` formatting · `refactor:` refactoring · `chore:` tooling

Standards

- Follow existing code style; meaningful names; comments only where the code can't speak
- Keep functions single-purpose; maintain TypeScript type safety

15. License

MIT License. Originally copyright ALLWEONE Team; this fork is a derivative work created for coursework purposes.